Table of Contents

- 8. Which sorting algorithm is not stable but can be used for in-place quick partitioning?
- 9. Which problem is a direct application of partitioning logic in Quick Sort?
- 10. For sorting 0s, 1s, and 2s in-place in a single pass, how many pointers are typically used?
- 1. How do you check if a string is a palindrome ignoring non-alphanumeric characters and case?
- 2. What is the optimal time complexity to find the smallest window in S containing all characters of T?
- 3. To determine if a string can be constructed by repeating a substring, which method is effective?
- 4. Which algorithm is best for finding all exact matches of a short pattern in a large text efficiently?
- 5. What is the output of compressing the string "aaabbbcc" using run-length encoding?
- 6. What is the time complexity of Rabin-Karp algorithm for string matching in the average case?
- 7. What is the role of the Z-function in string algorithms?
- 8. What's the main advantage of KMP over brute-force string matching?
- 9. To parse and evaluate "3 + 2 * 2" as an expression with precedence, which data structure is useful?
- 10. Which of the following is not true about Z-Algorithm?
- 1. Which approach is best for finding the longest substring with all unique characters?
- 2. What is the time complexity of sliding window approach for longest unique-character substring problem?
- 3. To count the number of distinct characters in every window of size K, which data structure is commonly used?
- 4. What is the time complexity of finding max sum of subarrays of size K using the sliding window technique?
- 5. Which condition indicates that a shrinking window is needed while finding the longest substring with all unique characters?

- 6. What should you do when a duplicate character is encountered in the sliding window for unique substring problems?
- 7. Which of the following is NOT a typical use-case of the sliding window technique?
- 8. What is the main benefit of using a sliding window over brute force in subarray problems?
- 9. In sliding window problems, the window is typically defined using:
- 10. Which data structure is ideal for tracking the frequency of characters in a current sliding window?
- 1. What is the base case in a recursive binary search on a sorted array?
- 2. What is the time complexity of generating all permutations of a string with length n?
- 3. How many total moves are required to solve the Towers of Hanoi for n disks?
- 4. Which type of recursion is used in the Towers of Hanoi problem?
- 5. What is the return condition (base case) when generating all permutations of a string?
- 6. In recursive binary search, what happens after comparing the middle element with the target?
- 7. Which of the following is not a type of recursion?
- 8. What is the call stack used for during recursion?
- 9. What is the primary risk of using deep recursion without a base case?
- 10. Which of the following problems cannot be solved efficiently without recursion or a stack-based approach?
- 1. What is the total number of subsets (power set) of a set with n elements?
- 2. In the N-Queens problem, what is the main condition to check before placing a queen?
- 3. What is the base case for generating subsets (power set) using recursion?
- 4. What is the time complexity of solving the N-Queens problem using backtracking?
- 5. In backtracking, what does the term "backtrack" mean?
- 6. In the "Rat in a Maze" problem, what approach is typically used?

- 7. When solving "Combination Sum" using recursion and backtracking, what condition must be met to add a combination to the result?
- 8. Which data structure is implicitly used to manage recursive calls in backtracking?
- 9. What makes a backtracking solution different from brute-force?
- 10. What is a necessary step when backtracking after trying a decision?
- 1. In the word search problem on a 2D board, what is the primary technique used?
- 2. While searching for a word in a grid (adjacent cells), what should be done after visiting a cell?
- 3. In the word search grid, how many directions are typically allowed for movement?
- 4. In a grid with obstacles (represented as 1s), what is the condition to move to the next cell?
- 5. What is the time complexity of solving a Sudoku board using backtracking?
- 6. In Sudoku backtracking, what must be validated before placing a digit in a cell?
- 7. Which technique ensures that the backtracking algorithm doesn't revisit already explored Sudoku states?
- 8. In path-counting with obstacles using recursion, what should be returned at a blocked cell?
- 9. In backtracking, why do we restore the previous state after a recursive call?
- 10. In word search and Sudoku, what data structure is commonly used to track used/visited cells?
- 1. Which algorithm is commonly used to detect a cycle in a linked list?
- 2. Once a cycle is detected using Floyd's algorithm, how can you find the starting node of the cycle?
- 3. What is the time complexity of merging two sorted linked lists into one sorted list?
- 4. Which of the following is the best method to check if a linked list is a palindrome?
- 5. What is the space complexity of checking if a linked list is a palindrome using optimal method?
- 6. To remove the N-th node from the end of a list in one pass, what technique is used?

- 7. What is the initial gap between fast and slow pointers when removing N-th node from the end?
- 8. After merging two sorted lists, which node should be chosen as the head?
- 9. Which condition indicates a palindrome in a linked list after reversing the second half?
- 10. What happens if we forget to set the next pointer of the last node to null when removing a cycle?
- 1. Which data structure is ideal to validate a string with parentheses, brackets, and braces?
- 2. What is the time complexity of checking for valid parentheses using a stack?
- 3. In a Min Stack, how is the minimum element retrieved in constant time?
- 4. What is the time complexity for push(), pop(), and getMin() in a well-designed Min Stack?
- 5. In the histogram problem, what does the stack keep track of?
- 6. What is the time complexity of finding the largest rectangle area in a histogram using a stack?
- 7. In the Next Greater Element problem on a circular array, what stack-based trick is used?
- 8. What is returned when no next greater element is found in a circular array?
- 9. What order should elements be processed for Next Greater Element using stack?
- 10. Which stack operation helps in backtracking during parentheses validation?
- 1. What is the order of nodes visited in an inorder traversal of a binary tree?
- 2. What data structure is commonly used to implement level order traversal (BFS) of a binary tree?
- 3. How is the maximum depth of a binary tree calculated recursively?
- 4. In a BST, what property helps find the lowest common ancestor (LCA) efficiently?
- 5. How do you generate all root-to-leaf paths in a binary tree?
- 6. Given preorder and inorder traversals, how do you identify the root of the tree?

- 7. What is the time complexity of constructing a binary tree from preorder and inorder traversals?
- 8. How is the k-th smallest element in a BST found?
- 9. Which traversal method produces sorted output for a BST?
- 10. What is the space complexity of recursive inorder traversal of a binary tree?
- 1. To find the longest substring with at most K distinct characters, which data structure is most suitable?
- 2. What is the main idea to find the smallest substring in s containing all characters of t?
- 3. When finding the longest substring without repeating characters, what key operation is done on the hash map?
- 4. What is the time complexity of finding the longest substring without repeating characters using a sliding window and hash map?
- 5. To find the total number of subarrays whose sum equals k, which technique is used?
- 6. In the subarray sum equals k problem, what does the hash map store?
- 7. How do you handle decrementing character counts when shrinking the sliding window in substring problems?
- 8. What is the space complexity for sliding window substring problems using hash maps?
- 9. In the minimum window substring problem, when do you move the left pointer?
- 10. Why is a hash map preferred over an array for substring problems involving character frequency?
- 1. What data structure is most efficient to find the kth largest element in an unsorted array?
- 2. To merge k sorted linked lists efficiently, which data structure is used?
- 3. What is the time complexity of merging k sorted lists with a priority queue?
- 4. How do you find the k most frequent elements in an array?
- 5. In the Activity Selection problem, which greedy strategy is optimal?
- 6. Huffman Coding builds what type of tree?

- 7. In Huffman Coding, what is the criteria to combine two nodes?
- 8. What is the main idea behind greedy algorithms?
- 9. What data structure is used to implement Huffman Coding efficiently?
- 10. What is the space complexity for merging k sorted lists using a priority queue?
- 1. Given an array where every element appears twice except one, which operation helps find the unique element?
- 2. What does Brian Kernighan's algorithm do to count set bits in an integer?
- 3. How do you reverse the bits of an integer?
- 4. To count how many times the number 1 appears in binary representations of all numbers from 1 to n, which approach is efficient?
- 5. How many subsets (power set) does a set of n elements have?
- 6. Which operation can be used to generate all subsets using bit masking?
- 7. What is the time complexity of counting set bits using Brian Kernighan's algorithm on a number with k set bits?
- 8. Which bitwise operation removes the rightmost set bit of a number x?
- 9. What is the result of XOR-ing a number with itself?
- 10. In bit manipulation, what does a mask with a single set bit (like 1 << i) represent?
- DP - 1: Company-Oriented DP Problems
- DP - 2: DP Patterns & Multi-dimensional DP
- DP - 3: DP on Trees
- Graphs
- Tries

Here are multiple-choice questions (MCQs) on Arrays and Advanced Array Algorithms (Frequency Arrays, Prefix Arrays, Two Pointers) and related problems:

---

1. Which of the following data structures is most suitable for checking if two strings are anagrams?

A. Stack
B. Queue
C. Hash Map or Frequency Array
D. Linked List

✅ Answer: C

---

2. After preprocessing an array for mode queries, what data structure is commonly used to answer in constant time?

A. Segment Tree
B. Hash Map + Prefix Sum Table
C. Mo's Algorithm
D. Range Mode Query Structure (with precomputed frequency)

✅ Answer: D

---

3. What is the time complexity to compute the sum of elements in a range [L, R] using prefix sum array?

A. O(1)
B. O(n)
C. O(log n)
D. O(R - L)

✅ Answer: A

---

4. Which technique is best suited for solving maximum sum subarray problem?

A. Prefix Arrays
B. Two Pointers
C. Sliding Window
D. Kadane's Algorithm

✅ Answer: D

---

5. What is the best time complexity to find all pairs that sum up to a target in an unsorted array?

A. $O(n^2)$
B. O(n log n)
C. O(n) with Hash Set
D. O(log n)

✅ Answer: C

---

6. Which approach is ideal for problems involving palindromes or symmetric subarrays?

A. Prefix Sum
B. Two Pointers (Opposite Ends)
C. Binary Search
D. Hash Map

✅ Answer: B

---

7. What is the space complexity of prefix sum array for an array of size n?

A. O(1)
B. O(n)
C. $O(n^2)$
D. O(log n)

✅ Answer: B

---

8. In the two-pointer technique, which of the following is a common use-case?

A. Reversing a linked list
B. Sorting an array
C. Finding a pair with given sum in a sorted array
D. Finding frequency of elements

✅ Answer: C

---

9. Given an array, how many operations are needed to preprocess it for prefix sum queries?

A. $O(n^2)$
B. O(n log n)
C. O(n)
D. O(1)

✅ Answer: C

---

10. What is the optimal way to find the number of subarrays with a given sum k?

A. Brute-force with nested loops
B. Two pointers
C. Hash Map with Prefix Sum
D. Binary Search

✅ Answer: C

Let me know if you want these in quiz format or need code-based MCQs.

Here are MCQs for Arrays and Advanced Array Algorithms - 2: Advanced Problems on Sorting:

---

1. What is the average time complexity of Quickselect algorithm used to find the kth smallest/largest element?

A. O(n log n)
B. $O(n^2)$
C. O(n)
D. O(log n)

✅ Answer: C

---

2. Which algorithm is best for sorting an array of colors with values {0, 1, 2}?

A. Merge Sort
B. Counting Sort
C. Dutch National Flag Algorithm
D. Quick Sort

✅ Answer: C

---

3. What is the time complexity of the Dutch National Flag Algorithm?

A. $O(n^2)$
B. O(n log n)
C. O(n)
D. O(1)

✅ Answer: C

---

4. You are given a sorted array and a target sum. What technique efficiently finds two elements that add up to the target?

A. Hash Map
B. Binary Search
C. Two Pointers
D. Sliding Window

✅ Answer: C

---

5. Which of the following can be used to solve the "kth largest element in an array" problem efficiently for large k?

A. Merge Sort
B. Min Heap
C. Hash Map
D. Bucket Sort

✅ Answer: B

---

6. What is the space complexity of Dutch National Flag Algorithm?

A. O(1)
B. O(n)
C. O(log n)
D. $O(n^2)$

✅ Answer: A

---

7. In a sorted array, if you're using two pointers to find a target sum, what is the best time complexity?

A. $O(n^2)$
B. O(n log n)
C. O(n)
D. O(log n)

✅ Answer: C

---

8. Which sorting algorithm is not stable but can be used for in-place quick partitioning?

A. Merge Sort
B. Quick Sort
C. Counting Sort
D. Bubble Sort

✅ Answer: B

---

9. Which problem is a direct application of partitioning logic in Quick Sort?

A. Merge Intervals
B. Kth Largest Element
C. Subarray Sum
D. Longest Increasing Subsequence

✅ Answer: B

---

10. For sorting 0s, 1s, and 2s in-place in a single pass, how many pointers are typically used?

A. 1
B. 2
C. 3
D. 4

✅ Answer: C

Let me know if you want explanations, code-based MCQs, or more practice questions.

Here are MCQs on Advanced String Algorithms, including two-pointer string problems, string matching, and implementation-based problems:

---

1. How do you check if a string is a palindrome ignoring non-alphanumeric characters and case?

A. Use Stack and Queue
B. Convert to lowercase and reverse
C. Two pointers with character checks
D. Regex and replace

✅ Answer: C

---

2. What is the optimal time complexity to find the smallest window in `S` containing all characters of `T`?

A. $O(n^2)$
B. O(n log n)
C. O(n + m) using Sliding Window + Frequency Count
D. $O(m^2)$

✅ Answer: C

---

3. To determine if a string can be constructed by repeating a substring, which method is effective?

A. Check all substrings
B. KMP Prefix Table
C. Rabin-Karp
D. Brute-force pattern matching

✅ Answer: B

---

4. Which algorithm is best for finding all exact matches of a short pattern in a large text efficiently?

A. Merge Sort
B. DFS
C. KMP or Rabin-Karp
D. BFS

✅ Answer: C

5. What is the output of compressing the string `"aaabbbcc"` using run-length encoding?

A. abc
B. a3b3c2
C. aa3bb3cc2
D. aab3bc3

✅ Answer: B

---

6. What is the time complexity of Rabin-Karp algorithm for string matching in the average case?

A. O(nm)
B. O(n + m)
C. O(n)
D. O(n + m) with hash comparisons

✅ Answer: D

---

7. What is the role of the Z-function in string algorithms?

A. Finds suffixes only
B. Finds longest prefix matching from each position
C. Counts all substrings
D. Matches suffixes and prefixes

✅ Answer: B

---

8. What's the main advantage of KMP over brute-force string matching?

A. Less memory
B. Skips re-checking previously matched characters
C. Faster average-case only
D. Always works in O(1)

✅ Answer: B

---

9. To parse and evaluate `"3 + 2 * 2"` as an expression with precedence, which data structure is useful?

A. Stack
B. Queue
C. Heap
D. Tree

✅ Answer: A

---

10. Which of the following is not true about Z-Algorithm?

A. It precomputes prefix matches
B. Time complexity is O(n)
C. It can be used for pattern matching
D. It uses binary search internally

✅ Answer: D

Let me know if you want hands-on coding MCQs, quizzes, or topic-wise PDF.

Here are MCQs on Sliding Window: Advanced Problems, covering unique substrings, counting conditions in windows, and fixed-size/max-sum subarrays:

---

1. Which approach is best for finding the longest substring with all unique characters?

A. Brute force
B. Two nested loops
C. Sliding Window with HashSet
D. Binary Search

✅ Answer: C

---

2. What is the time complexity of sliding window approach for longest unique-character substring problem?

A. $O(n^2)$
B. O(n)
C. O(n log n)
D. O(log n)

✅ Answer: B

---

3. To count the number of distinct characters in every window of size K, which data structure is commonly used?

A. Stack
B. Queue
C. Hash Map
D. Heap

✅ Answer: C

---

4. What is the time complexity of finding max sum of subarrays of size K using the sliding window technique?

A. $O(n^2)$
B. O(n log n)
C. O(n)
D. O(k log n)

✅ Answer: C

---

5. Which condition indicates that a shrinking window is needed while finding the longest substring with all unique characters?

A. Window size exceeds K
B. Duplicate character is found
C. Window has a negative sum
D. Total characters exceed array size

✅ Answer: B

---

6. What should you do when a duplicate character is encountered in the sliding window for unique substring problems?

A. Stop
B. Expand window further
C. Shrink window from left until duplicate is removed
D. Restart the algorithm

✅ Answer: C

7. Which of the following is NOT a typical use-case of the sliding window technique?

A. Finding maximum element in a window
B. Finding unique characters in a substring
C. Finding the minimum spanning tree
D. Calculating averages in a fixed-size window

✅ Answer: C

---

8. What is the main benefit of using a sliding window over brute force in subarray problems?

A. More accurate
B. Less space
C. Reduces time complexity from $O(n^2)$ to $O(n)$
D. Avoids integer overflow

✅ Answer: C

---

9. In sliding window problems, the window is typically defined using:

A. A fixed queue
B. Two pointers (start and end)
C. Binary search boundaries
D. Linked list traversal

✅ Answer: B

---

10. Which data structure is ideal for tracking the frequency of characters in a current sliding window?

A. Stack
B. Heap
C. Hash Map or Array
D. Set

✅ Answer: C

Let me know if you want coding-based MCQs or practice questions with explanations.

Here are MCQs on Recursion, covering basics, types, and key problems like binary search, permutations, and Towers of Hanoi:

---

1. What is the base case in a recursive binary search on a sorted array?

A. When mid equals end
B. When low > high
C. When target equals array[0]
D. When the array is empty

✅ Answer: B

---

2. What is the time complexity of generating all permutations of a string with length `n`?

A. O(n)
B. $O(n^2)$
C. O(n!)
D. $O(2^n)$

✅ Answer: C

---

3. How many total moves are required to solve the Towers of Hanoi for `n` disks?

A. n
B. 2^n
C. $2^n - 1$
D. n^2

✅ Answer: C

---

4. Which type of recursion is used in the Towers of Hanoi problem?

A. Linear Recursion
B. Tail Recursion
C. Tree Recursion
D. Iterative Recursion

✅ Answer: C

5. What is the return condition (base case) when generating all permutations of a string?

A. When the string is sorted
B. When the string contains all unique characters
C. When the current index equals the string length
D. When no more swaps can be made

✅ Answer: C

---

6. In recursive binary search, what happens after comparing the middle element with the target?

A. Recurse into both halves
B. Recurse into the half that may contain the target
C. Exit immediately
D. Loop through the array

✅ Answer: B

---

7. Which of the following is not a type of recursion?

A. Tail Recursion
B. Head Recursion
C. Middle Recursion
D. Tree Recursion

✅ Answer: C

---

8. What is the call stack used for during recursion?

A. To manage arrays
B. To store recursive function parameters and return addresses
C. To sort elements
D. To store final answers

✅ Answer: B

---

9. What is the primary risk of using deep recursion without a base case?

A. Low accuracy
B. Stack Overflow

C. Memory leak
D. Slow execution

✅ Answer: B

---

10. Which of the following problems cannot be solved efficiently without recursion or a stack-based approach?

A. Binary Search
B. Fibonacci Sequence
C. Generating Permutations
D. Calculating Average

✅ Answer: C

Let me know if you'd like coding MCQs, trace-based questions, or a visual explanation of recursion.

Here are MCQs on Recursion and Backtracking, focused on implementation, call stacks, and classic backtracking problems like power set, N-Queens, rat in a maze, and combination sum:

---

1. What is the total number of subsets (power set) of a set with `n` elements?

A. 2n
B. n^2
C. n!
D. 2^n

✅ Answer: D

---

2. In the N-Queens problem, what is the main condition to check before placing a queen?

A. Same row
B. Same column
C. Same diagonal
D. All of the above

✅ Answer: D

3. What is the base case for generating subsets (power set) using recursion?

A. When the subset size is even
B. When the current index reaches the end of the array
C. When the sum of subset is zero
D. When the subset is a palindrome

✅ Answer: B

---

4. What is the time complexity of solving the N-Queens problem using backtracking?

A. O(n)
B. $O(n^2)$
C. O(n!)
D. $O(2^n)$

✅ Answer: C

---

5. In backtracking, what does the term "backtrack" mean?

A. Restart the algorithm
B. Skip unnecessary branches
C. Remove the last choice and try another
D. Store all results in a stack

✅ Answer: C

---

6. In the "Rat in a Maze" problem, what approach is typically used?

A. BFS
B. DFS with backtracking
C. Dynamic Programming
D. Greedy Algorithm

✅ Answer: B

---

7. When solving "Combination Sum" using recursion and backtracking, what condition must be met to add a

combination to the result?

A. All numbers are unique
B. Combination size is even
C. Sum of elements equals target
D. The array is sorted

✅ Answer: C

---

8. Which data structure is implicitly used to manage recursive calls in backtracking?

A. Queue
B. Heap
C. Call Stack
D. Hash Table

✅ Answer: C

---

9. What makes a backtracking solution different from brute-force?

A. It doesn't explore all possible options
B. It prunes paths that cannot possibly lead to a solution
C. It uses sorting
D. It only works on sorted inputs

✅ Answer: B

---

10. What is a necessary step when backtracking after trying a decision?

A. Print the current path
B. Revert the state (undo the decision)
C. Push to queue
D. Exit the program

✅ Answer: B

Let me know if you want trace-based dry-run questions, code MCQs, or visual diagrams of backtracking trees.

Here are MCQs on Advanced Backtracking and Recursion on Matrices, covering grid-based search, word construction, unique paths with obstacles, and Sudoku solving:

---

1. In the word search problem on a 2D board, what is the primary technique used?

A. BFS
B. DFS with backtracking
C. Greedy
D. Dynamic Programming

✅ Answer: B

---

2. While searching for a word in a grid (adjacent cells), what should be done after visiting a cell?

A. Skip to next row
B. Mark it as visited
C. Change all neighbors
D. Sort characters

✅ Answer: B

---

3. In the word search grid, how many directions are typically allowed for movement?

A. 2 (up/down)
B. 4 (up/down/left/right)
C. 6
D. 8

✅ Answer: B

---

4. In a grid with obstacles (represented as 1s), what is the condition to move to the next cell?

A. If it's visited
B. If it's a 1
C. If it's a 0 (no obstacle)
D. If the row is even

✅ Answer: C

5. What is the time complexity of solving a Sudoku board using backtracking?

A. O(n)
B. $O(9^{82})$
C. O(9!)
D. O(1)

✅ Answer: B

---

6. In Sudoku backtracking, what must be validated before placing a digit in a cell?

A. Row uniqueness
B. Column uniqueness
C. 3x3 grid uniqueness
D. All of the above

✅ Answer: D

---

7. Which technique ensures that the backtracking algorithm doesn't revisit already explored Sudoku states?

A. Visited matrix
B. Hash set for rows, columns, and boxes
C. Queue
D. Sorting

✅ Answer: B

---

8. In path-counting with obstacles using recursion, what should be returned at a blocked cell?

A. 1
B. 0
C. The cell value
D. Total paths from neighbors

✅ Answer: B

---

9. In backtracking, why do we restore the previous state after a recursive call?

A. To save memory
B. To allow other recursive paths to try different options
C. To speed up execution
D. To exit early

✅ Answer: B

---

10. In word search and Sudoku, what data structure is commonly used to track used/visited cells?

A. Stack
B. Priority Queue
C. 2D Boolean array
D. Linked List

✅ Answer: C

Let me know if you'd like dry-run walkthroughs, diagrams, or MCQs with code snippets.

Here are MCQs on Linked List: Interview-Based Problems, covering cycle detection/removal, merging sorted lists, palindrome check, and one-pass node deletion:

---

1. Which algorithm is commonly used to detect a cycle in a linked list?

A. BFS
B. Floyd's Tortoise and Hare (two pointers)
C. DFS
D. KMP Algorithm

✅ Answer: B

---

2. Once a cycle is detected using Floyd's algorithm, how can you find the starting node of the cycle?

A. Restart one pointer from head and move both one step at a time
B. Reverse the list

C. Count nodes

D. Use a stack

✅ Answer: A

---

3. What is the time complexity of merging two sorted linked lists into one sorted list?

A. O(1)

B. O(n + m)

C. O(nm)

D. O(log n)

✅ Answer: B

---

4. Which of the following is the best method to check if a linked list is a palindrome?

A. Convert to array and check

B. Reverse entire list

C. Reverse second half and compare with first

D. Sort and check

✅ Answer: C

---

5. What is the space complexity of checking if a linked list is a palindrome using optimal method?

A. O(n)

B. O(log n)

C. O(1)

D. $O(n^2)$

✅ Answer: C

---

6. To remove the N-th node from the end of a list in one pass, what technique is used?

A. Stack

B. Two pointers (fast and slow)

C. Recursion

D. Sorting

✅ Answer: B

7. What is the initial gap between fast and slow pointers when removing N-th node from the end?

A. 0
B. 1
C. N
D. N+1

✅ Answer: C

---

8. After merging two sorted lists, which node should be chosen as the head?

A. The one with the largest value
B. The one with the smallest value
C. Any of the two
D. None

✅ Answer: B

---

9. Which condition indicates a palindrome in a linked list after reversing the second half?

A. Head == tail
B. First and second halves are equal node by node
C. Length is even
D. Middle node is null

✅ Answer: B

---

10. What happens if we forget to set the `next` pointer of the last node to `null` when removing a cycle?

A. Nothing
B. List remains circular
C. List becomes empty
D. An exception is thrown

✅ Answer: B

Let me know if you'd like MCQs with trace-based dry-runs or code-based multiple choice.

Here are MCQs on Stacks: Interview-Based Problems, covering valid parentheses, min-stack design, histogram area, and next greater element in a circular array:

---

1. Which data structure is ideal to validate a string with parentheses, brackets, and braces?

A. Queue
B. Set
C. Stack
D. Hash Map

✅ Answer: C

---

2. What is the time complexity of checking for valid parentheses using a stack?

A. $O(n^2)$
B. O(n)
C. O(log n)
D. O(1)

✅ Answer: B

---

3. In a Min Stack, how is the minimum element retrieved in constant time?

A. Use a priority queue
B. Keep a variable with the min
C. Use an auxiliary stack storing minimums
D. Sort stack after each push

✅ Answer: C

---

4. What is the time complexity for `push()`, `pop()`, and `getMin()` in a well-designed Min Stack?

A. O(n)
B. O(log n)
C. O(n log n)
D. O(1)

✅ Answer: D

---

5. In the histogram problem, what does the stack keep track of?

A. Bar heights
B. Rectangle widths
C. Indices of bars in increasing height
D. Total area

✅ Answer: C

---

6. What is the time complexity of finding the largest rectangle area in a histogram using a stack?

A. O(n)
B. $O(n^2)$
C. O(log n)
D. O(n log n)

✅ Answer: A

---

7. In the Next Greater Element problem on a circular array, what stack-based trick is used?

A. Double traverse the array
B. Reverse the array
C. Sort the array
D. Use two stacks

✅ Answer: A

---

8. What is returned when no next greater element is found in a circular array?

A. 0
B. -1
C. 1
D. Infinity

✅ Answer: B

---

9. What order should elements be processed for Next Greater Element using stack?

A. From right to left
B. From left to right
C. In sorted order
D. In reverse sorted order

✅ Answer: A

---

10. Which stack operation helps in backtracking during parentheses validation?

A. Push
B. Pop
C. Top
D. GetMin

✅ Answer: B

Let me know if you'd like dry-run based MCQs, code MCQs, or practice problems.

Here are MCQs on Binary Trees and BSTs, covering traversal, depth, LCA, tree construction, and k-th smallest element:

---

1. What is the order of nodes visited in an inorder traversal of a binary tree?

A. Root → Left → Right
B. Left → Root → Right
C. Left → Right → Root
D. Right → Left → Root

✅ Answer: B

---

2. What data structure is commonly used to implement level order traversal (BFS) of a binary tree?

A. Stack
B. Queue

C. Hash Map
D. Priority Queue

✅ Answer: B

---

3. How is the maximum depth of a binary tree calculated recursively?

A. Max depth of left + max depth of right
B. Min depth of left or right + 1
C. Max depth of left or right + 1
D. Sum of depths of all nodes

✅ Answer: C

---

4. In a BST, what property helps find the lowest common ancestor (LCA) efficiently?

A. Left subtree values < root < right subtree values
B. Height balanced property
C. Same values in both subtrees
D. Complete binary tree structure

✅ Answer: A

---

5. How do you generate all root-to-leaf paths in a binary tree?

A. Use BFS and track parents
B. Use DFS and build path string during recursion
C. Sort nodes first
D. Use level order traversal

✅ Answer: B

---

6. Given preorder and inorder traversals, how do you identify the root of the tree?

A. First element in preorder
B. First element in inorder
C. Last element in preorder
D. Last element in inorder

✅ Answer: A

7. What is the time complexity of constructing a binary tree from preorder and inorder traversals?

A. O(n)
B. $O(n^2)$
C. O(log n)
D. O(n log n)

✅ Answer: B (without optimization; O(n) with hash maps)

---

8. How is the k-th smallest element in a BST found?

A. Level order traversal
B. Inorder traversal
C. Postorder traversal
D. Preorder traversal

✅ Answer: B

---

9. Which traversal method produces sorted output for a BST?

A. Preorder
B. Postorder
C. Inorder
D. Level order

✅ Answer: C

---

10. What is the space complexity of recursive inorder traversal of a binary tree?

A. O(1)
B. O(h), where h is tree height
C. O(n)
D. O(log n)

✅ Answer: B

If you want trace-based MCQs or coding challenges on trees, just ask!

Here are MCQs on Hash Maps: Advanced Hashing Techniques & Interview Problems, covering substrings with distinct chars, minimum window substring, longest substring without repeats, and subarray sums:

---

1. To find the longest substring with at most K distinct characters, which data structure is most suitable?

A. Stack
B. Hash Map
C. Queue
D. Linked List

✅ Answer: B

---

2. What is the main idea to find the smallest substring in `s` containing all characters of `t`?

A. Sliding window with two pointers and character frequency count
B. Sort both strings and compare
C. Brute force all substrings
D. Use stack to match characters

✅ Answer: A

---

3. When finding the longest substring without repeating characters, what key operation is done on the hash map?

A. Store character frequencies
B. Store last seen index of characters
C. Store character priorities
D. Store character counts only for vowels

✅ Answer: B

---

4. What is the time complexity of finding the longest substring without repeating characters using a sliding window and hash map?

A. $O(n^2)$
B. O(n log n)

C. O(n)

D. O(log n)

✅ Answer: C

---

5. To find the total number of subarrays whose sum equals k, which technique is used?

A. Two pointers with sorting

B. Prefix sum with hash map storing frequencies

C. Brute force checking all subarrays

D. Divide and conquer

✅ Answer: B

---

6. In the subarray sum equals k problem, what does the hash map store?

A. Sum → Number of times this sum has occurred

B. Index → Value

C. Value → Index

D. Characters → Frequency

✅ Answer: A

---

7. How do you handle decrementing character counts when shrinking the sliding window in substring problems?

A. Remove characters from hash map

B. Update count and remove key if zero

C. Keep counts unchanged

D. Clear hash map and restart

✅ Answer: B

---

8. What is the space complexity for sliding window substring problems using hash maps?

A. O(1)

B. O(k) where k is the number of distinct characters

C. $O(n^2)$

D. O(log n)

✅ Answer: B

9. In the minimum window substring problem, when do you move the left pointer?

A. When the current window contains all characters of `t`
B. Always after right pointer moves
C. When no characters match
D. Never

✅ Answer: A

---

10. Why is a hash map preferred over an array for substring problems involving character frequency?

A. Arrays are slower
B. Hash map can handle any character set, not just limited alphabets
C. Arrays use more memory
D. Hash maps sort characters automatically

✅ Answer: B

If you want coding MCQs or dry-run trace problems on hashing, just ask!

Here are MCQs on Priority Queue and Greedy Algorithms, covering kth largest element, merging k lists, frequent elements, activity selection, and Huffman coding:

---

1. What data structure is most efficient to find the kth largest element in an unsorted array?

A. Max Heap
B. Min Heap
C. Stack
D. Queue

✅ Answer: B
(Min heap of size k is used to track kth largest)

---

2. To merge k sorted linked lists efficiently, which data structure is used?

A. Stack
B. Priority Queue (Min Heap)
C. Hash Map
D. Queue

✅ Answer: B

---

3. What is the time complexity of merging k sorted lists with a priority queue?

A. O(n log n)
B. O(nk)
C. O(n log k)
D. O(k log n)

✅ Answer: C
(n = total nodes, k = number of lists)

---

4. How do you find the k most frequent elements in an array?

A. Sort elements by frequency
B. Use a hash map + min heap of size k
C. Use BFS
D. Use two pointers

✅ Answer: B

---

5. In the Activity Selection problem, which greedy strategy is optimal?

A. Select activities with earliest start time
B. Select activities with shortest duration
C. Select activities with earliest finish time
D. Select activities with latest start time

✅ Answer: C

---

6. Huffman Coding builds what type of tree?

A. Binary Search Tree
B. Prefix-free Binary Tree

C. AVL Tree

D. Trie

✅ Answer: B

---

7. In Huffman Coding, what is the criteria to combine two nodes?

A. Combine two nodes with the largest frequencies

B. Combine two nodes with the smallest frequencies

C. Combine nodes randomly

D. Combine nodes with maximum depth

✅ Answer: B

---

8. What is the main idea behind greedy algorithms?

A. Make globally optimal choice at each step

B. Backtrack to find solution

C. Use recursion

D. Check all permutations

✅ Answer: A

---

9. What data structure is used to implement Huffman Coding efficiently?

A. Stack

B. Min Heap (Priority Queue)

C. Hash Map

D. Queue

✅ Answer: B

---

10. What is the space complexity for merging k sorted lists using a priority queue?

A. O(n)

B. O(k)

C. O(1)

D. O(log n)

✅ Answer: B

If you want MCQs with explanation or code snippet-based questions, let me know!

Here are MCQs on Bit-Masking & Bit Manipulation covering single unique element, counting set bits, bit reversal, counting 1s in 1 to n, and subsets generation:

---

1. Given an array where every element appears twice except one, which operation helps find the unique element?

A. Addition
B. XOR
C. AND
D. OR

✅ Answer: B

---

2. What does Brian Kernighan's algorithm do to count set bits in an integer?

A. Iterates through all bits
B. Repeatedly flips the rightmost set bit to zero until the number becomes zero
C. Uses division
D. Uses string conversion

✅ Answer: B

---

3. How do you reverse the bits of an integer?

A. By using XOR operations only
B. By swapping bits from both ends moving inward
C. By using addition and subtraction
D. By sorting bits

✅ Answer: B

---

4. To count how many times the number 1 appears in binary representations of all numbers from 1 to n, which approach is efficient?

A. Check each number individually
B. Use a mathematical formula or DP approach based on bit positions
C. Use string matching
D. Use recursion only

✅ Answer: B

---

5. How many subsets (power set) does a set of n elements have?

A. n
B. 2^n
C. n^2
D. n!

✅ Answer: B

---

6. Which operation can be used to generate all subsets using bit masking?

A. Iterate from 0 to 2^n - 1 and treat each number as a bitmask representing subset inclusion
B. Iterate over elements sequentially
C. Use DFS only
D. Use sorting

✅ Answer: A

---

7. What is the time complexity of counting set bits using Brian Kernighan's algorithm on a number with k set bits?

A. O(n) where n is total bits
B. O(k) where k is number of set bits
C. O(1)
D. O(n^2)

✅ Answer: B

---

8. Which bitwise operation removes the rightmost set bit of a number `x`?

A. `x & (x-1)`
B. `x | (x-1)`
C. `x ^ (x-1)`
D. `x << 1`

✅ Answer: A

---

9. What is the result of XOR-ing a number with itself?

A. The number itself
B. 1
C. 0
D. Undefined

✅ Answer: C

---

10. In bit manipulation, what does a mask with a single set bit (like `1 << i`) represent?

A. The number with all bits set
B. The number with only the i-th bit set
C. The number with the i-th bit cleared
D. A random number

✅ Answer: B

If you want me to prepare detailed explanations or coding MCQs on bit manipulation, just say!

Here are MCQs on Dynamic Programming (DP) covering company-oriented problems, DP patterns, multidimensional DP, and DP on trees:

DP - 1: Company-Oriented DP Problems

1. The longest common substring between two strings can be found using which approach?
A. Hashing only
B. Dynamic Programming
C. Greedy Algorithm
D. BFS

✅ Answer: B

2. Which method can find the longest palindromic substring efficiently besides DP?
A. Expanding around centers
B. Sorting
C. Binary Search
D. BFS

✅ Answer: A

---

3. Finding the longest substring that appears at least twice is often solved using?
A. Trie
B. Suffix Arrays or Suffix Trees
C. Priority Queues
D. DP only

✅ Answer: B

---

4. Edit distance between two strings measures?
A. Number of common characters
B. Minimum number of operations to convert one string into the other
C. Maximum length of common substring
D. Longest palindromic subsequence length

✅ Answer: B

DP - 2: DP Patterns & Multi-dimensional DP

5. Which DP problem allows fractions of items to be taken for maximum value?
A. 0-1 Knapsack
B. Fractional Knapsack
C. Subset Sum
D. Coin Change

✅ Answer: B

---

6. Longest common subsequence between two strings is solved using?
A. Greedy

B. DP
C. Hashing
D. Binary Search

✅ Answer: B

---

7. The minimum number of coins required to make a given amount is solved using?
A. Backtracking
B. DP
C. BFS
D. Sorting

✅ Answer: B

---

8. Which of the following is NOT a valid edit operation in the edit distance problem?
A. Insertion
B. Deletion
C. Substitution
D. Rotation

✅ Answer: D

DP - 3: DP on Trees

9. The diameter of a binary tree is?
A. The height of the tree
B. The length of the longest path between any two nodes
C. The number of nodes
D. The number of leaves

✅ Answer: B

---

10. To find the maximum sum path in a binary tree that can start and end anywhere, which technique is used?
A. Greedy
B. DP on Trees (Postorder traversal with memoization)
C. BFS
D. Sorting

✅ Answer: B

---

11. The time complexity of computing the longest increasing subsequence using DP is?
A. O(n)
B. O(n log n)
C. $O(n^2)$
D. O(log n)

✅ Answer: C

---

12. Matching patterns with wildcards (*) and (?) is solved using?
A. DP
B. Greedy
C. Backtracking only
D. Hashing

✅ Answer: A

If you want me to prepare coding MCQs, complexity analysis, or problem variations on these DP topics, just ask!

Here are MCQs on Graphs and Tries covering graph representations, BFS, DFS, cycle detection, shortest paths, MST, topological sort, and Tries:

Graphs

1. Which data structure is commonly used to represent a graph efficiently for BFS and DFS?
A. Adjacency Matrix
B. Adjacency List
C. Hash Map only
D. Linked List

✅ Answer: B

---

2. BFS traversal in a graph is used to find?
A. Longest path
B. Shortest path in an unweighted graph
C. Minimum spanning tree
D. Cycle detection

✅ Answer: B

---

3. What is the main difference between BFS and DFS in graph traversal?
A. BFS uses recursion, DFS uses queue
B. BFS uses queue, DFS uses recursion/stack
C. BFS works only on trees, DFS on graphs
D. BFS is faster than DFS always

✅ Answer: B

---

4. How can you detect a cycle in a directed graph?
A. Using BFS with a queue
B. Using DFS with a recursion stack
C. Using topological sort only
D. Using Dijkstra's Algorithm

✅ Answer: B

---

5. Which algorithm is used to find the shortest path from a source node in a graph with non-negative weights?
A. Bellman-Ford
B. Dijkstra's Algorithm
C. Floyd-Warshall
D. Kruskal's Algorithm

✅ Answer: B

---

6. Bellman-Ford algorithm can handle graphs with?
A. Negative weight edges
B. Negative cycles
C. Only positive weights
D. Only unweighted edges

✅ Answer: A

---

7. What is the purpose of Topological Sort?
A. Detect cycles in undirected graph
B. Find shortest path in graph

C. Order nodes in Directed Acyclic Graph (DAG) linearly
D. Find connected components

✅ Answer: C

---

8. Which algorithm finds the Minimum Spanning Tree (MST) in a connected, undirected weighted graph?
A. Dijkstra's
B. Bellman-Ford
C. Kruskal's Algorithm
D. Floyd-Warshall

✅ Answer: C

Tries

9. What is a Trie used for?
A. Fast string search and prefix matching
B. Graph traversal
C. Finding minimum spanning tree
D. Sorting numbers

✅ Answer: A

---

10. What is the time complexity to search for a word in a Trie of size N?
A. O(N) where N is number of words
B. O(length of the word)
C. O(log N)
D. $O(N^2)$

✅ Answer: B

---

11. What kind of queries can Tries efficiently handle?
A. Range queries on integers
B. Prefix queries and autocomplete
C. Cycle detection
D. Shortest path

✅ Answer: B

---

12. Which of the following is NOT a property of Trie nodes?

A. Contains children pointers for each character

B. Stores the complete word at each node

C. May have a flag to indicate end of a word

D. Supports fast prefix search

✅ Answer: B

If you want coding problems, implementations, or step-by-step walkthroughs for any of these graph or trie topics, just ask!